

Toollery: Scaling LLM Agents to Hundreds of Skills/Tools

Anonymous ACL submission

Abstract

As LLM agents are exposed to hundreds to tens of thousands of skills, tools, and API functions, full-library prompting becomes costly, slow, and less reliable: each added candidate increases prompt tokens and latency, while longer candidate lists introduce more distractors for LLM selection. We introduce **Toollery**, a training-free candidate-compression framework for scalable LLM skill/tool selection. Toollery generates user-intent queries from each skill/tool specification and builds a retrieval index that maps real user requests to compact candidate sets before final LLM decision-making. By treating high-level skills and atomic tools as selectable capabilities, Toollery can be applied to both skill libraries and tool registries. We evaluate Toollery on SkillsBench with roughly 79K skills, BFCL-V4 with over 440 atomic tools, and a proprietary 220-intent vehicle-control dataset. Across these settings, Toollery keeps online selection bounded to a compact top- k candidate set, improves recall over ordinary specification retrieval, and improves BFCL-V4 end-to-end tool-selection accuracy compared with full-library prompting. These results support Toollery as a practical framework for large and evolving agent capability libraries: shift semantic matching offline, keep online prompts compact, and let the final LLM decide only among a compact set of retrieved candidates.

1 Introduction

LLM agents increasingly rely on external resources to complete user requests: a resource may be a high-level procedural skill, an MCP service, an atomic tool, or an API function (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024). Although these resources differ in granularity and execution interface, they play the same role before execution: the agent must decide whether each

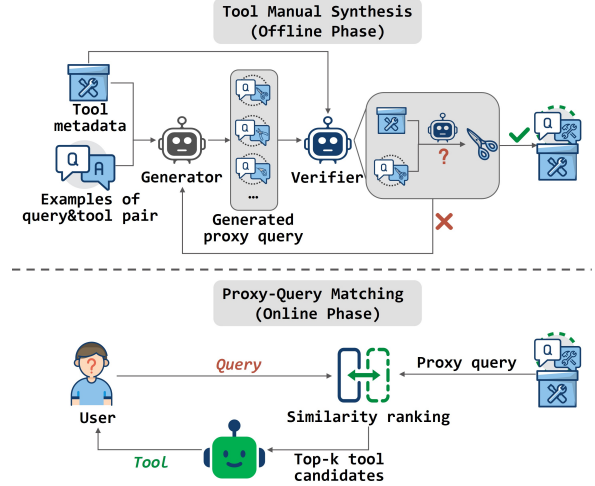


Figure 1: **Overview of Toollery.** Toollery scales LLM skill/tool selection by moving part of intent matching offline. For each skill/tool specification, a generator produces user-intent queries that can be mapped back to the skill or tool. At inference time, Toollery retrieves a compact candidate set from these user-intent queries and specifications, and passes only these candidates to the final LLM for decision making.

one should be exposed to the final LLM for the current user request. We therefore treat them as selectable *capabilities* throughout this paper. As capability libraries grow, skill/tool selection becomes a production bottleneck. Industrial reports and our own agent deployments show that loading skill/tool specifications upfront can consume tens of thousands of tokens before any user work begins, while selection accuracy degrades as the library grows (Anthropic, 2026, 2025). In our deployment, after loading more than 200 skills and tools, even a trivial greeting can consume more than 50K input tokens and over 5 seconds of response latency. Scalable LLM skill/tool selection is therefore necessary for controlling token cost, latency, and accuracy as public ecosystems aggregate increasingly large capability libraries (Open-

Claw, 2026; Lin et al., 2025; Mo et al., 2025; Xu et al., 2025).

The common practice is full-library prompting: place every skill/tool specification in the prompt and let the LLM choose. This preserves the full decision context, but makes the agent pay the cost of reviewing every available capability on every request, even when the request needs no tool. As a result, easy or irrelevant requests still incur high prompt cost, higher latency, and more distractors for final LLM decision making. Practical agents therefore need a candidate-compression layer that narrows the action space before LLM inference.

Retrieval is a natural way to implement candidate compression, but ordinary specification retrieval is limited. Existing retrieval-based methods commonly retrieve candidates directly from skill/tool specifications, including names, descriptions, and schemas (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026). This assumes that user requests resemble specifications. In practice, users describe goals, constraints, or partial tasks, while specifications describe capabilities, parameters, and developer-facing usage details. This intent-specification mismatch is especially problematic for heterogeneous libraries whose capabilities are written at different levels of abstraction.

We introduce **Toollery**, a training-free framework that frames scalable LLM skill/tool selection as **intent-level retrieval**. Rather than retrieving over specifications directly, Toollery asks what kinds of user requests should select each capability. It treats high-level skills, MCP services, atomic tools, and API functions as selectable capabilities under the same abstraction. Toollery generates user-intent queries from each capabilities specification, retrieves a compact top- k candidate set using these intent-augmented representations, and forwards only the selected candidates to the final LLM. This separates library-scale matching from schema-aware final LLM decision making.

Our experiments show that Toollery turns the growing skill/tool-selection bottleneck into a bounded retrieval problem. On SkillsBench, BFCL-V4, and a proprietary vehicle-control dataset, candidate pools range from hundreds of tools or intents to roughly 79K skills, yet Toollery keeps online LLM selection bounded to a compact top- k candidate set. This reduces the number of specifications exposed to the final LLM, keeping token cost and latency controllable as the library grows. At the same time, the retrieved top- k sets

achieve higher recall than ordinary specification retrieval, and BFCL-V4 end-to-end experiments show that Toollery improves final tool-selection accuracy compared with full-candidate prompting.

This paper makes the following contributions:

- We formulate skill/tool selection as intent-level candidate compression, treating high-level skills, MCP services, atomic tools, and API functions as selectable capabilities under the same abstraction.
- We present Toollery, a training-free framework that shifts semantic matching offline by generating user-intent queries from capability specifications and retrieving compact top- k candidate sets before final LLM decision making
- We evaluate Toollery across large-scale skill routing, public atomic-tool benchmarks, and a proprietary multi-intent dataset, showing that Toollery bounds online LLM selection by limiting the number of skill/tool specifications exposed to the final LLM, thereby controlling prompt tokens, latency, and distractor exposure while improving retrieval recall and end-to-end tool-selection accuracy under candidate-pool growth.

2 Related Work

Prior work improves LLM tool use by training or evaluating models that invoke external tools once relevant tools are available in context or training data (Yao et al., 2022; Schick et al., 2023; Patil et al., 2024; Qin et al., 2023; Li et al., 2023; Patil et al., 2025). Toollery addresses the complementary deployment problem of selecting a compact candidate set from a large skill library before final LLM decision making.

Recent retrieval and routing methods reduce the action space before tool selection (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026; Gan and Sun, 2025; Paramanayakam et al., 2025). MassTool learns search-based tool retrieval signals, SkillRouter retrieves and reranks skills at scale, and RAG-MCP-style systems retrieve tool documentation to reduce prompt bloat. Toollery is complementary: it trains no router, and expands each searchable skill document with generated user intents so standard retrievers can match user phrasing to developer-facing metadata.

A key distinction is that Toollery treats high-level skills, MCP services, atomic tools, and API functions as the same type of selectable capability. This unified abstraction lets the Toollery operate over heterogeneous capability libraries: generated intents are generated according to each capability specification, retrieval compresses the library to a compact candidate set, and the final LLM performs schema-aware selection only over retrieved candidates.

3 The Toollery Design

Toollery separates full-library search from final LLM decision-making. Figure 1 shows the overall pipeline: Toollery first generates user-intent queries for each selectable capability, then uses a standard retriever at inference time to retrieve a compact candidate set for the final LLM. Throughout the paper, we use *skill* as the general unit of selection: a skill may be a high-level procedural capability, an MCP service, an atomic tool, or an API function. Toollery can retrieve either over generated user-intent queries alone or over skill/tool metadata augmented with those generated queries; the latter preserves both user-facing intent signals and developer-facing specification details.

3.1 Problem Formulation

Let $\mathcal{A} = \{a_1, \dots, a_M\}$ denote the full library of available capabilities, where each capability may be a high-level skill, an MCP service, an atomic tool, or an API function. Each capability a_i has a specification s_i , which may include its name, description, schema, instruction, usage constraints, examples, or documentation. Given a user request x , the goal is to construct a compact candidate set

$$\mathcal{C}_k(x) \subset \mathcal{A}, \quad |\mathcal{C}_k(x)| = k \ll M, \quad (1)$$

that contains the capability or capabilities needed to satisfy the request. The final LLM then receives x and the specifications of only the candidates in $\mathcal{C}_k(x)$, and produces the final selection and any required arguments.

The main difficulty is the mismatch between user requests and specifications. Users describe goals, constraints, or partial tasks, while specifications describe capabilities, parameters, and usage details. Toollery addresses this mismatch by constructing a manual for the full library: a set of generated user-intent queries that describe when each capability should be selected.

3.2 Manual Construction

For each capability a_i , a generator model G produces N user-intent queries from its specification:

$$Q_i = G(s_i; N, \pi), \quad (2)$$

where π prompts the generator to write realistic user requests that would require a_i . The manual for the full library is the union of the generated user-intent queries for all capabilities:

$$\mathcal{M} = \{(q, a_i) : q \in Q_i, a_i \in \mathcal{A}\}. \quad (3)$$

Each manual entry stores a generated user-intent query and the identifier of the capability whose specification produced it.

Toollery can optionally apply source-capability recovery filtering to remove generated user-intent queries that are not discriminative under competition. For each generated query $q \in Q_i$, Toollery constructs a verification pool

$$\mathcal{P}_{i,q} = \{a_i\} \cup \mathcal{D}_{i,q}, \quad (4)$$

where $\mathcal{D}_{i,q} \subset \mathcal{A} \setminus \{a_i\}$ is a set of random distractor capabilities. A verifier model H receives q and the specifications of capabilities in $\mathcal{P}_{i,q}$, then selects the best matching capability or abstains:

$$\hat{a} = H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}). \quad (5)$$

Toollery retains q only when the verifier recovers the source capability, $\hat{a} = a_i$. This generation and filtering step is offline, so its cost is amortized across future requests and only affected capabilities need updates when the full library changes.

3.3 Retrieval over the Manual

At inference time, Toollery retrieves candidates using the user request x and the manual $\mathcal{M}$. The retriever is not tied to a specific implementation: it can be instantiated with sparse lexical retrieval such as BM25, dense retrieval, or a RAG-style retriever. Retrieval can score generated user-intent queries in the manual directly, or score specifications augmented with the corresponding user-intent queries from the manual. Both are Toollery retrieval settings because both use the manual constructed from the full library; they differ only in whether the retriever also sees the original specification text.

When retrieval is performed over manual entries, multiple retrieved user-intent queries may

point to the same capability. Toollery therefore aggregates query-level evidence into capability-level scores. Let $f(x, q)$ be the retrieval score between user request x and a generated user-intent query q . Toollery ranks each capability by aggregating the strongest matching queries associated with that capability:

$$S(x, a_i) = \text{Agg}\{f(x, q) : (q, a_i) \in \mathcal{M}\}. \quad (6)$$

The candidate set is then

$$\mathcal{C}_k(x) = \text{TopK}_{a_i \in \mathcal{A}} S(x, a_i). \quad (7)$$

In our implementation, Agg averages the strongest retrieved query scores for each capability and adds a small evidence-count bonus; exact aggregation parameters are listed in Appendix B.

3.4 Candidate Compression and Final Selection

For each selected capability, Toollery instantiates a compact specification containing the fields required by the LLM final decision, such as the name, description, schema, and necessary instructions. The final LLM receives the original user request and the compact specifications for only $\mathcal{C}_k(x)$; it does not receive the full library or the manual. Retrieval therefore narrows the action space, while the LLM remains responsible for resolving residual ambiguity, selecting the final capability, and filling arguments.

This design changes the online cost from full-library prompting to subset prompting. Let M be the number of capabilities in the full library and let L_s be the average token length of a compact specification. Full-library prompting sends approximately $O(M \cdot L_s)$ specification tokens to the LLM, so prompt cost grows linearly as the library expands. Toollery instead sends only the retrieved top- k specifications, approximately $O(k \cdot L_s)$ tokens, where k is a deployment-time candidate budget and typically satisfies $k \ll M$. Because k can remain fixed even as M grows, the final LLM sees a bounded candidate set while the manual and retrieval index absorb library growth offline.

4 Experiments

We evaluate Toollery as a candidate-compression layer for scalable capability selection. The experiments ask three questions. First, can Toollery recover the correct capabilities from large libraries while keeping the online candidate set bounded?

Second, does the same manual-based retrieval design transfer from high-level skills to atomic API-style tools? Third, after retrieval, does candidate compression improve end-to-end LLM tool selection as the candidate pool grows?

4.1 Experimental Setup

Benchmarks. We evaluate Toollery on two public benchmarks and one proprietary industrial dataset. The first public benchmark is SkillsBench (Li et al., 2026; Zheng et al., 2026), which contains 75 evaluation queries: 24 single-skill queries and 51 multi-skill queries. Each user request is evaluated against an ~79K-skill pool. The second public benchmark is BFCL-V4 (Patil et al., 2024). We use its non-live and live multiple-tool instances and evaluate each request against the complete candidate pool, treating each tool or API function as an atomic capability. Finally, we evaluate on a proprietary vehicle-control dataset with 3,396 real user requests over a 220-intent production pool. Half of the requests contain two gold intents, and the other half contain three gold intents.

Settings and baselines. We compare retrieval settings that differ in what information is available to the retriever. *Raw specification* retrieves only from the original skill/tool specifications and does not use Toollery’s manual. Toollery first constructs a manual for the full library, where the manual contains generated user-intent queries for each capability. We evaluate two ways to retrieve with this manual: *Toollery (manual only)* retrieves from the manual alone, while *Toollery (manual + specification)* retrieves from specifications augmented with the corresponding user-intent queries in the manual. Both are Toollery settings because both use the generated manual; they differ only in whether the retriever also sees the original specification text. Within these settings, we instantiate retrieval with BM25 or RAGAnything.

Metrics and cost. For retrieval, we report Hit@1, Recall@10, Recall@50, MRR@10, nDCG@10, FullCoverage@10, and online retrieval latency as Avg ms/query. Hit@1 measures whether the top-ranked candidate is correct. Recall@10/50 measure whether gold capabilities appear in the retrieved candidate set. MRR@10 rewards methods that rank the first correct capability earlier in the top 10, while nDCG@10 evaluates the quality of the whole top-10 ranking with higher weight on earlier

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	309.4	0.553	0.571	0.726	0.625	0.512	0.413
Raw specification	RAGAnything	38210.6	0.600	0.568	0.712	0.663	0.500	0.393
Raw specification	SkillRouter [†]	–	0.740	0.704	–	0.791	–	0.520
Toollery (manual only)	BM25	218.9	0.840	0.880	0.951	0.881	0.846	0.760
Toollery (manual only)	RAGAnything	21505.9	0.900	0.922	0.953	0.925	0.882	0.827
Toollery (manual + specification)	BM25	623.0	0.920	0.942	0.968	0.942	0.880	0.853
Toollery (manual + specification)	RAGAnything	41070.5	0.933	0.900	0.950	0.943	0.843	0.827

Table 1: SkillsBench retrieval over roughly 79K skills, averaged over the easy and hard splits. Raw specification retrieves from original skill specifications only. Full-library prompting at this scale would require roughly 43.1M input tokens per request; Toollery top-10 prompting requires roughly 27.7K tokens, over three orders of magnitude smaller. [†] denotes the SkillRouter reference using SR-Emb-0.6B $\times$ SR-Rank-0.6B.

positions. FullCoverage@10 requires all gold capabilities for multi-capability requests to appear in the top 10. For the proprietary vehicle-control dataset, we additionally report Recall@20 to measure intermediate candidate-set coverage. For end-to-end experiments, we report final selection accuracy after the final LLM chooses from the retrieved subset. We report the offline manual construction protocol and cost in Appendix B; using official DeepSeek-V4-Pro prices as of June 9, 2026,¹ construction costs approximately \$0.22 per 100 skills on average. In the main tables, we report online retrieval latency as Avg ms/query to characterize the serving-time cost of candidate compression.

4.2 Skill Retrieval on SkillsBench

The SkillsBench experiment tests Toollery at the scale where full-library prompting is no longer a feasible serving strategy. Prompting the final LLM with the full ~79K-skill library would require roughly 43M input tokens per request, while Toollery top-10 prompting requires about 27.7K tokens, over three orders of magnitude smaller. The practical question is therefore whether retrieval can compress this full library into a small candidate set without losing the needed capabilities. Table 1 reports retrieval quality averaged over the easy and hard splits.

Toollery substantially improves this large-library retrieval step. With BM25, Toollery (manual + specification) improves Hit@1 from 0.553 to 0.920, Recall@10 from 0.571 to 0.942, and FullCoverage@10 from 0.413 to 0.853 over raw specification retrieval. This result supports the core claim that Toollery-based candidate compression can recover the needed capabilities from a very large library before final LLM decision-making. The latency column reports retrieval time only;

¹https://api-docs.deepseek.com/quick_start/pricing, accessed June 9, 2026.

with BM25, Toollery performs this ~79K-skill candidate compression in 623.0 ms/query.

The comparison also clarifies the role of the manual. Toollery (manual only) is already strong, showing that generated user-intent queries are useful retrieval targets by themselves. Toollery (manual + specification) further improves most metrics because the retriever can use both user-facing intent signals from the manual and developer-facing details from the original specifications.

4.3 Atomic-Tool Retrieval and End-to-End Selection on BFCL-V4

The BFCL-V4 experiments test whether the same Toollery design transfers from high-level skills to atomic tools and API functions, and whether candidate compression improves the final LLM selection step. Tables 2 and 3 evaluate retrieval over the complete BFCL-live 457-tool pool and BFCL non-live 443-tool pool, respectively. Unlike SkillsBench, exceeding the context window limit, full-library prompting for BFCL-V4 dataset is feasible, which lets us compare Toollery with prompting over all candidate tool specifications.

Toollery remains effective in this atomic-tool setting. On BFCL-live, Toollery (manual + specification) with RAGAnything reaches 0.776 Hit@1 and 0.980 Recall@10. On BFCL non-live, it reaches 0.845 Hit@1 and 1.000 Recall@10. Raw specification retrieval is already strong because tool specifications are short and structured, but Toollery still improves top- k recall by adding user-intent signals from the manual. This matters because the final LLM can only select the correct tool if retrieval keeps that tool inside the compact candidate set.

Table 4 reports BFCL-V4 end-to-end selection using DeepSeek V3.2 and shows this retrieval improvement translates into a better end-to-end serving tradeoff. On BFCL-live, full-library prompting exposes all 457 tool specifications to the fi-

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	3.6	0.643	0.931	0.976	0.744	0.790	0.931
Raw specification	RAGAnything	152.5	0.737	0.976	0.995	0.831	0.867	0.976
Toollery (manual only)	BM25	0.3	0.468	0.822	0.934	0.584	0.642	0.822
Toollery (manual only)	RAGAnything	81.8	0.543	0.855	0.929	0.653	0.702	0.855
Toollery (manual + specification)	BM25	5.0	0.731	0.948	0.986	0.811	0.845	0.948
Toollery (manual + specification)	RAGAnything	139.7	0.776	0.980	0.997	0.856	0.887	0.980

Table 2: BFCL-V4 live multiple retrieval over the complete 457-tool pool.

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	14.4	0.780	0.970	0.995	0.854	0.883	0.970
Raw specification	RAGAnything	524.6	0.825	1.000	1.000	0.895	0.922	1.000
Toollery (manual only)	BM25	0.2	0.555	0.885	0.990	0.668	0.721	0.885
Toollery (manual only)	RAGAnything	77.9	0.605	0.925	0.985	0.722	0.772	0.925
Toollery (manual + specification)	BM25	25.9	0.815	1.000	1.000	0.887	0.915	1.000
Toollery (manual + specification)	RAGAnything	657.3	0.845	1.000	1.000	0.904	0.928	1.000

Table 3: BFCL-V4 non-live multiple retrieval over the complete 443-tool pool.

Dataset	Selection Setting	# Candidates	Input Tokens	Latency	Accuracy
live	Full-library prompt	457	84.1K	11.8s	0.770
live	Toollery	10	2.1K	1.6s	0.831
non-live	Full-library prompt	443	57.0K	8.4s	0.703
non-live	Toollery	10	1.3K	1.5s	0.814

Table 4: BFCL-V4 end-to-end selection under full-library prompting and Toollery candidate compression. Full-library prompting exposes all tool specifications to the final LLM, while Toollery exposes only the retrieved top-10 candidate specifications. Accuracy is measured with DeepSeek V3.2 as the final selector.

nal LLM, requiring 84.1K input tokens and 11.8 seconds of latency with 0.770 accuracy. Toollery exposes only 10 retrieved specifications, reducing input tokens to 2.1K and latency to 1.6 seconds, while improving accuracy to 0.831. On BFCL non-live, Toollery similarly reduces input tokens from 57.0K to 1.3K and latency from 8.4 seconds to 1.5 seconds, while improving accuracy from 0.703 to 0.814.

These results directly support the candidate-compression view of Toollery. Full-library prompting makes the final LLM pay for every available tool specification and exposes it to more distractors. Toollery instead keeps online selection bounded: the manual and retrieval index absorb capabilities library growth offline, while the final LLM receives only a compact top- k set of candidate specifications.

4.4 Multi-Intent Retrieval on Proprietary Dataset

The proprietary vehicle-control dataset tests Toollery in a production-style multi-intent setting. Each request is a real user request mapped to two or three gold vehicle-control intents, and retrieval is performed over a complete 220-intent pool. Full-library prompting over this pool would require roughly 35K input tokens per request. This

is feasible for some long-context models, but it is still much larger than a top-10 Toollery prompt and would be paid on every request. Toollery reduces this to about 1.9K tokens with top-10 candidates; even with top-50 candidates, the prompt remains about 9.1K tokens.

Table 5 shows that Toollery improves candidate-set coverage for real multi-intent requests. Compared with raw BM25 specification retrieval, Toollery improves Recall@10 from 0.545 to 0.662 and Recall@50 from 0.867 to 0.924. With query decomposition, Toollery further improves Recall@10 to 0.684, Recall@20 to 0.803, Recall@50 to 0.926, and FullCoverage@10 to 0.381. These gains are important because multi-intent requests require the compressed candidate set to preserve more than one correct capability. The result suggests that Toollery can reduce the prompt size of a production capability library while still keeping the candidate set complete enough for downstream multi-intent selection.

5 Discussion and Conclusion

The experiments show that full-library prompting quickly becomes a poor serving strategy as capability libraries grow. At SkillsBench scale, it is infeasible: prompting over the full ~79K-skill library would require roughly 43.1M input tokens for every request. Even at smaller scales, the cost remains substantial: BFCL-V4 requires 57.0K-84.1K input tokens and 8.4-11.8 seconds of latency over 443-457 tools, while the proprietary 220-intent vehicle-control pool still requires roughly 35.2K input tokens. The deployment pressure is therefore structural: full-library prompting makes the final LLM pay for every available capability on every request, including trivial or irrelevant ones.

Retrieval Setting	Retriever	Online ms/query	Hit@1	Recall@10	Recall@20	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw specification	BM25	6.4	0.430	0.545	0.696	0.867	0.576	0.446	0.224
Raw specification	RAGAnything	144.5	0.408	0.582	0.718	0.871	0.588	0.464	0.229
Toollery (manual only)	BM25	0.7	0.481	0.670	0.784	0.918	0.648	0.544	0.358
Toollery (manual only)	RAGAnything	221.6	0.453	0.666	0.781	0.902	0.633	0.538	0.357
Toollery (manual + specification)	BM25	69.7	0.484	0.684	0.803	0.926	0.654	0.553	0.381
Toollery (manual + specification)	RAGAnything	253.4	0.403	0.630	0.758	0.893	0.591	0.500	0.316

Table 5: Proprietary vehicle-control retrieval over the complete 220-intent pool. Results are averaged over 3,396 real company-internal requests with two or three gold intents. Full-library prompting over this 220-intent library would require roughly 35K input tokens per request, which is feasible in terms of the context window limit but still much larger than a top-10 Toollery prompt. The proprietary dataset is used only for deployment-facing evaluation and is not released.

Toollery addresses this bottleneck by moving semantic matching out of the final prompt and into offline manual construction plus online retrieval. Across all three settings, Toollery keeps final selection bounded to a compact top- k candidate set: SkillsBench drops from roughly 43.1M full-library tokens to 27.7K top-10 tokens; BFCL-V4 drops from 84.1K to 2.1K tokens on live multiple and from 57.0K to 1.3K on non-live multiple; and the proprietary dataset drops from 35.2K to 1.9K top-10 tokens. On BFCL-V4, these token reductions also translate into latency reductions from 11.8s to 1.6s and from 8.4s to 1.5s.

The accuracy results show that Toollery does not buy efficiency by sacrificing selection quality. Manual-based retrieval improves over raw specification retrieval across SkillsBench, BFCL-V4, and the proprietary multi-intent dataset, and improves candidate-set coverage for multi-capability requests. In BFCL-V4 end-to-end selection, Toollery also improves final tool-selection accuracy over full-library prompting, suggesting that candidate compression reduces distractors as well as prompt cost and latency.

Taken together, these results suggest a different design principle for scalable agents: skill/tool selection should not be implemented as prompt engineering over an ever-growing full library. The final LLM should not pay the token, latency, and distractor cost of every available capability on every request. Instead, large-library selection should include an intermediate candidate-compression layer. Toollery instantiates this principle by constructing a manual from capability specifications offline, retrieving a compact candidate set online, and letting the final LLM perform schema-aware decision-making only over retrieved specifications. Because this interface is retrieval-based, Toollery remains compatible with different retrievers and different final LLMs, while applying to high-level skills, MCP services, atomic tools, and API func-

tions under the same capability abstraction.

The remaining deployment tradeoff is where to pay the semantic matching cost. Toollery moves part of this cost to offline manual construction, which must be updated when capabilities are added or changed. This cost is amortized across future requests, while the savings in final-prompt tokens and latency occur on every online request. For large and evolving agent systems, this is the favorable side of the tradeoff: the library can grow offline, while online selection remains bounded. Overall, Toollery turns large-library skill/tool selection from an unbounded full-prompting problem into a practical retrieval-and-selection pipeline.

6 Limitations

Toollery depends on the coverage and precision of its generated queries. If the generator fails to produce realistic user intents for a skill, retrieval recall is bounded no matter which retrieval backend is used. Conversely, if generated queries are too broad, they can add noisy matches to the query-augmented document. The generated-query-only ablation uses source-skill recovery filtering to reduce this risk, but the filter does not provide a formal correctness guarantee.

The framework also adds offline maintenance cost. New or modified skills require query generation, representation, and index updates. Optional filtering adds further cost when a deployment uses generated-query-only retrieval. Our offline construction cost quantifies this cost for the SkillsBench easy and hard pool. This cost is amortized across future requests, but it may matter in rapidly changing skill ecosystems where tools are frequently added, removed, or rewritten. Deployments should therefore track construction cost separately from online retrieval and final-selection latency.

Finally, Toollery addresses candidate selection, not full agent execution. It does not solve multi-step planning, tool sequencing, execution monitoring, recovery from failed calls, or argument correctness by itself. In multi-step agents, Toollery can reduce the candidate set at each decision point, but the broader planning policy and execution controller remain outside the scope of this work.

References

- Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.
- Anthropic. 2025. Introducing Advanced Tool Use on the Claude Developer Platform. <https://www.anthropic.com/engineering/advanced-tool-use>. Published: 2025-11-24; accessed: 2026-06-09.
- Anthropic. 2026. Tool Search Tool. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>. Accessed: 2026-06-09.
- Tiantian Gan and Qiyao Sun. 2025. Rag-mcp: Mitigating prompt bloat in llm tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore. Association for Computational Linguistics.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, and 1 others. 2026. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*.
- Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- OpenClaw. 2026. ClawHub: Skill and Plugin Registry for OpenClaw. <https://github.com/openclaw/clawhub>. Accessed: 2026-06-09.
- Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.

- 654 Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and
655 Zhijie Deng. 2025. Tps-bench: Evaluating ai agents’
656 tool planning\& scheduling abilities in compound-
657 ing tasks. *arXiv preprint arXiv:2511.01527*.
- 658 Shunyu Yao, Howard Chen, John Yang, and Karthik
659 Narasimhan. 2022. Webshop: Towards scalable
660 real-world web interaction with grounded language
661 agents. *Advances in Neural Information Processing*
662 *Systems*, 35:20744–20757.
- 663 Hanjie Zheng, Chenglin Li, Yibing Liu, Jiazhan Feng,
664 Wenhao Wu, Jiabao Guo, Yicheng Peng, Tian-
665 hang Li, Qian Zhang, Chi Zhang, and 1 others.
666 2026. Skillrouter: Retrieve-and-rerank skill se-
667 lection for llm agents at scale. *arXiv preprint*
668 *arXiv:2603.22455*.

A Additional End-to-End Results

Tables 6 and 7 report BFCL-V4 live and non-live multiple end-to-end selection accuracy under increasing candidate-pool sizes. The original non-live multiple split contains 200 requests and 443 unique tools, while the live multiple split contains 1,053 requests and 457 unique tools. The final column is therefore the complete candidate pool for each split, not a 1,000-candidate setting. The columns compare final selection when the model sees all candidates in the sampled pool against selection after Toollery retrieves a compact subset. We include both prompt-based selection and function-calling selection to test whether candidate compression helps across different final decision interfaces.

B Reproducibility and Deployment Details

B.1 Offline Construction Cost

To make the offline cost explicit, we measure query generation and recovery filtering on the SkillsBench easy and hard splits using DeepSeek-V4-Pro as both generator and verifier. Each skill receives three generated candidate queries and each non-empty candidate is checked against the source skill plus eight random distractors. Table 9 reports the measured offline construction cost for the full 78,361-skill easy pool and 79,141-skill hard pool. Under official DeepSeek-V4-Pro cache-miss input and output pricing,² this corresponds to approximately \$0.22 per 100 skills on average. For the easy split, construction produces 235K generated candidates, 228K retained queries, 361.8M total tokens, and approximately \$168.70; for the hard split, the corresponding values are 237K candidates, 230K retained queries, 365.4M tokens, and \$170.38. These values describe offline construction only; the online latency tables use warmed indexes and exclude this one-time generation and filtering cost.

²https://api-docs.deepseek.com/quick_start/pricing/, accessed June 2, 2026.

Table 6: BFCL-V4 live multiple results

Model	≤ 10	20	30	40	50	100	457
Qwen3-4B (Prompt)	85.50	86.00	88.00	82.50	78.00	69.50	24.88
Qwen3-4B (Prompt) + Toollery	87.00 $\uparrow$	84.50	87.00	86.50 $\uparrow$	82.50 $\uparrow$	81.00 $\uparrow$	83.75 $\uparrow$
Qwen3-8B (Prompt)	97.00	93.50	92.00	84.00	83.00	84.50	38.50
Qwen3-8B (Prompt) + Toollery	95.50	98.50 $\uparrow$	98.50 $\uparrow$	92.50 $\uparrow$	94.50	86.50 $\uparrow$	79.38 $\uparrow$
DeepSeek V3.2 (Prompt)	94.50	80.00	84.00	89.50	79.50	63.00	77.00
DeepSeek V3.2 (Prompt) + Toollery	98.00 $\uparrow$	78.50	94.50 $\uparrow$	99.50 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	83.13 $\uparrow$
GPT-5-mini (Prompt)	83.00	93.00	90.00	87.00	88.00	88.00	79.75
GPT-5-mini (Prompt) + Toollery	83.50 $\uparrow$	94.00 $\uparrow$	89.50	95.00 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	84.50 $\uparrow$
GPT-5-mini (FC)	67.00	62.00	63.50	62.00	61.00	63.50	57.00
GPT-5-mini (FC) + Toollery	74.50 $\uparrow$	67.50 $\uparrow$	69.00 $\uparrow$	69.50 $\uparrow$	67.00 $\uparrow$	67.50 $\uparrow$	66.00 $\uparrow$
Qwen3-8B (FC)	93.50	94.50	92.50	83.00	83.50	83.00	60.00
Qwen3-8B (FC) + Toollery	93.50	98.00 $\uparrow$	90.50	99.00 $\uparrow$	91.50 $\uparrow$	88.50 $\uparrow$	85.88 $\uparrow$
Qwen3-235B (FC)	98.00	93.00	87.50	89.50	85.50	80.00	81.13
Qwen3-235B (FC) + Toollery	98.50 $\uparrow$	92.50	89.00 $\uparrow$	89.50	90.00	91.50 $\uparrow$	88.63 $\uparrow$

Table 7: BFCL-V4 non-live multiple results

Model	≤ 10	20	30	40	50	100	443
Qwen3-4B (Prompt)	95.50	99.50	97.50	96.50	95.50	90.00	24.25
Qwen3-4B (Prompt) + Toollery	100.00 $\uparrow$	97.50	98.00 $\uparrow$	95.00	98.00 $\uparrow$	91.50 $\uparrow$	83.63 $\uparrow$
Qwen3-8B (Prompt)	97.50	98.50	99.50	97.50	97.00	89.50	37.63
Qwen3-8B (Prompt) + Toollery	99.50 $\uparrow$	100.00 $\uparrow$	97.50	96.00	98.00 $\uparrow$	91.00 $\uparrow$	80.63 $\uparrow$
DeepSeek V3.2 (Prompt)	86.00	99.50	97.00	94.50	96.50	85.50	70.25
DeepSeek V3.2 (Prompt) + Toollery	97.50 $\uparrow$	97.50	95.00	98.00 $\uparrow$	98.50 $\uparrow$	89.50 $\uparrow$	81.38 $\uparrow$
GPT-5-mini (Prompt)	88.00	87.50	92.50	85.00	88.50	88.00	80.00
GPT-5-mini (Prompt) + Toollery	88.00	89.50 $\uparrow$	91.50	86.50 $\uparrow$	95.00 $\uparrow$	90.00 $\uparrow$	82.00 $\uparrow$
GPT-5-mini (FC)	79.50	69.50	65.50	72.00	70.50	63.50	58.63
GPT-5-mini (FC) + Toollery	81.50 $\uparrow$	77.00 $\uparrow$	75.50 $\uparrow$	73.00 $\uparrow$	76.00 $\uparrow$	69.50 $\uparrow$	68.63 $\uparrow$
Qwen3-8B (FC)	97.50	97.50	97.50	98.50	96.00	85.00	60.75
Qwen3-8B (FC) + Toollery	99.50 $\uparrow$	96.50	97.50	96.50	97.00 $\uparrow$	90.50 $\uparrow$	86.75 $\uparrow$
Qwen3-235B (FC)	95.50	99.50	96.50	96.00	97.50	80.00	79.13
Qwen3-235B (FC) + Toollery	96.00 $\uparrow$	99.00	98.50 $\uparrow$	98.00 $\uparrow$	98.00 $\uparrow$	93.50 $\uparrow$	94.00 $\uparrow$

Item	Value used in reported runs
SkillsBench pools	78,361 easy skills; 79,141 hard skills; 75 requests per split
BFCL-live end-to-end pool	457 tools; 1,053 live multiple requests; final column uses the complete pool
BFCL-non-live end-to-end pool	443 tools; 200 multiple requests; final column uses the complete pool
Generated-query decoding	DeepSeek-V4-Pro through an OpenAI-compatible API; 3 generated queries per skill/tool; seed 31; 4 workers; batch size 4
Verifier prompt	asks DeepSeek-V4-Pro to choose one matching skill/tool id from the target plus distractors, or NONE if no candidate fits
Distractor construction	verifier pool uses the target plus 8 randomly sampled non-gold distractors; scaled candidate pools sample the gold tool plus non-gold tools without replacement
BM25 configuration	local tokenizer in <code>toollery.text</code> ; Okapi BM25 with $k_1 = 1.5, b = 0.75$; query-augmented documents concatenate name, category, description, schema, and generated queries
RAGAnything configuration	text-only RAGAnything/LightRAG cache with one compact chunk per skill/tool; retrieval queries LightRAG chunk vectors and recovers tool ids from returned context order
Retrieval embedding model	Qwen/Qwen3-Embedding-0.6B for neural retrieval and latency replay
Aggregation parameters	top- $c = 3$ averaged query scores with evidence bonus $b = 5, \lambda = 0.05$
Candidate budgets	top-50 for retrieval-quality tables; BFCL-live efficiency diagnostics use top-3 retrieved candidates; end-to-end table columns denote the all-candidate pool size before Toollery compression
Latency protocol	warmed indexes/caches; reports online retrieval or final-selection latency only; excludes offline query generation, verification, embedding, and index construction
Offline-cost	SkillsBench easy and hard split; DeepSeek-V4-Pro generator and verifier; 3 generated queries per skill; target plus 8 random distractors for verification; cost computed with cache-miss input pricing
Release plan	prompts, normalized generated-query artifacts, run configurations, and metric scripts will be released with the anonymized artifact

Table 8: Reproducibility details for the main retrieval and efficiency results. Secrets and local API endpoints are omitted; paths in the released run configs identify the exact output artifacts.

Setting	Skills	Cand.	Ret.	Tok.	Cost
SkillsBench easy	78,361	235K	228K	361.8M	\$168.70
SkillsBench hard	79,141	237K	230K	365.4M	\$170.38

Table 9: Offline construction cost for generated-query construction and verifier filtering. The measured rows use the SkillsBench easy and hard splits with three generated queries per skill. Cost uses official DeepSeek-V4-Pro pricing: \$0.435/M cache-miss input tokens and \$0.87/M output tokens; cache-hit discounts are not applied.